第 17 章 传输层安全

0. 学习目标总览

教材给出本章六个学习目标：

1. 小结Web安全威胁和Web流量安全方法。
2. 概述传输层安全（TLS）。
3. 理解安全套接层和传输层安全的不同。
4. 比较传输层安全中使用的伪随机函数和本书之前讨论的函数。
5. 概述HTTPS（SSL上的HTTP）。
6. 概述SSH（Secure Shell）。

本章主线是把密码机制放到 TCP/IP 通信路径中：

```
Web 威胁
  |
  |-- 安全部署位置：IP 层 / TLS 传输层 / 应用层
  |
  |-- TLS
  |     |-- 记录协议：分段、压缩、MAC、加密
  |     |-- 握手：认证、协商、密钥生成
  |     `-- HTTPS：HTTP 作为 TLS 应用数据
  |
  `-- SSH
        |-- 传输层：服务器认证、保密性、完整性
        |-- 用户认证
        `-- 连接层：多信道与端口转发
```

1. 学习目标 1：Web 安全威胁和流量安全方法

1.1 Web 的安全特点

Web 是运行在因特网和 TCP/IP 内联网上的客户端/服务器应用。它需要专门安全工具，主要因为：

- 浏览器、服务器和内容开发看似简单，但底层软件复杂，潜藏漏洞；
- Web 服务器通常连接企业内部系统，服务器被攻陷可能扩大为整个站点的入侵；
- 普通用户安全意识有限，往往不了解服务背后的安全风险。

1.2 Web 威胁类别

安全目标	典型威胁	可能后果	常用对策
完整性	修改用户数据、特洛伊木马浏览器、修改内存或传输中消息	信息丢失、机器受损、引入更多攻击	加密校验和、完整性保护
保密性	网络窃听、窃取服务器或客户端数据、窃取配置和通信内容	信息泄露、隐私丧失	加密、Web 代理
可用性	破坏用户线程、伪造消息使机器溢出、填满磁盘或内存、DNS 攻击	中断、干扰、无法正常工作	难以完全防止，需检测和恢复

安全目标	典型威胁	可能后果	常用对策
认证	冒充合法用户、伪造数据	用户作出错误决定、相信虚假信息	加密认证技术

被动攻击主要窃听通信并获取受限权限；主动攻击则伪装身份、修改消息或网站信息。按位置还可分为服务器、浏览器和浏览器—服务器通信链路上的威胁。本章重点是通信安全。

1.3 安全部署位置

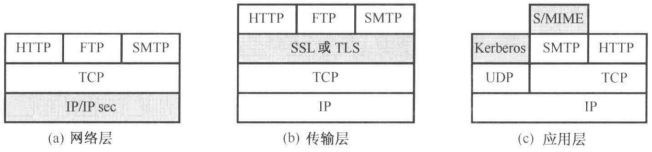


图 17.1　安全设施在 TCP/IP 协议栈中的相对位置

图 17.1 安全设施在 TCP/IP 协议栈中的相对位置

位置	例子	特点
网络层	IPsec	对终端用户和应用透明，可形成通用方案并过滤选定流量。
传输层	SSL/TLS	可作为协议栈的一部分对应用透明，也可嵌入浏览器和服务器软件。
应用层	S/MIME、Kerberos、应用专用安全	可按具体应用需求定制，但每个应用需分别实现安全功能。

2. 学习目标 2：传输层安全 TLS

2.1 TLS 架构、会话与连接

TLS 是从 SSL 发展而来的 Internet 标准，依赖 TCP 提供可靠的端到端安全服务。SSL 仍可能存在于旧系统中，但已被 IETF 和多数 TLS 实现弃用。

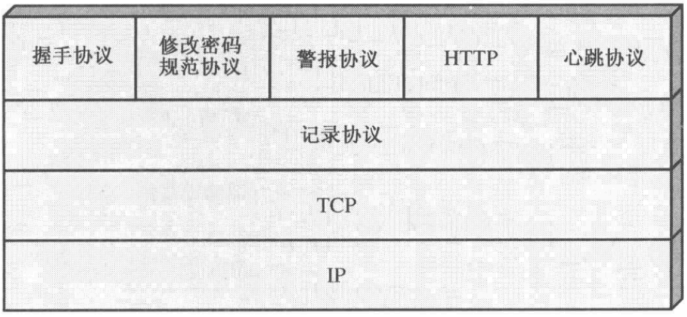


图 17.2　TLS 协议栈

图 17.2 TLS 协议栈

TLS 由记录协议和三个高层协议组成：握手协议、修改密码规范协议、警报协议；心跳协议在独立 RFC 中定义。HTTP 等应用协议在 TLS 顶部运行。

> **关于高层协议与记录协议与 TCP 关系**
>
> 它们的互动不是“握手协议主动去找记录协议”，而是**严格的“向下调用、向上分发”模式**，通过**函数接口**和**头部标记**完成：
>
> **互动过程（发送方向）**：
>
> 1. 高层协议（比如握手协议）生成一条完整的消息（如 `ClientHello`）。
> 2. 高层协议**调用**记录协议提供的接口（可以理解为调用一个名为 `Record_Send(数据类型, 数据内容)` 的函数），把数据内容传给它，并告诉它“这是握手数据”。
> 3. 记录协议拿到数据后，**不关心内容是什么**，只负责把它当作一段普通的字节流，进行分段、压缩（可选）、贴上MAC、加密（如果已协商）。
> 4. 记录协议给处理完的数据**加上固定的5字节明文头部**，其中最关键的就是**内容类型（Content Type）**字段——把高层传下来的类型（握手=22，警报=21等）写入该字段。
> 5. 组装成完整的“TLS记录”交给TCP。
>
> **互动过程（接收方向）**：
>
> 1. TCP把收到的字节流交给记录协议。
> 2. 记录协议解析出5字节头部，解密并验证完整性，还原出原始数据。
> 3. 记录协议**查看头部里的“内容类型”字段**。
> 4. 如果类型是22，记录协议就**把数据向上分发给“握手协议”的处理函数**；如果类型是23，就分发给“HTTP”的处理函数。

概念	含义
TLS 连接	对等实体之间短暂的传输关系；每个连接关联一个会话。
TLS 会话	由握手创建的客户端—服务器关联，定义可由多个连接共享的密码安全参数，避免每次连接都重新协商。
当前状态	已建立会话后实际用于读写的状态。
挂起状态	握手期间创建，握手完成后复制为当前读写状态。

会话状态包括**会话标识**、**对等实体证书** X.509 v 3 、**压缩方法**、**密码规范**、48 字节**主密钥**和是否**可恢复**标志。连接状态包括**双方随机数**、**双方写 MAC 密钥**、**双方写加密密钥**、CBC 的 **IV** 和从 0 开始递增的**序号**；序号不能超过 $2-1$。

> **连接与会话**
>
> **会话（Session）** 是客户端和服务器之间的一个逻辑关联，由握手协议创建，其本质是一组可被多个连接共享的密码安全参数，包括会话标识、主密钥（Master Secret）、证书、压缩方法和密码规范。会话的创建开销较大，其设计目的是避免为每次通信重新执行完整的密钥协商过程。
>
> **连接（Connection）** 则是基于某个会话所建立的实际传输通道，它代表一个短暂的对等关系，提供适合于传输的服务。每个连接都独立维护自身运行状态，包括双方各自的随机数、读写MAC密钥、读

写加密密钥、初始化向量和用于防重放的序号。

二者的核心关系是**一对多的复用**：一个会话可以关联多个连接，但每个连接在任意时刻都只归属于一个会话。连接的生命周期通常较短，可以随数据传输完成而终止，而会话可以保持较长时间以支持后续新连接的快速恢复，从而将昂贵的密码学计算开销（如Diffie-Hellman密钥交换或RSA解密）分摊到多个连接上。

一次连接是一个持久化的逻辑通道，需要经过完整的握手协商（如TCP三次握手和TLS密钥协商）来建立，并维持双方的状态（如序列号、加密密钥、窗口大小等）；而**一次报文发送**只是在这个已有通道内进行的一次数据传输操作。

2.2 TLS 记录协议

记录协议向上层提供保密性和消息完整性。发送方向依次执行：

```
应用消息
  |-- 分段（每段不超过 2^14 字节）
  |-- 可选无损压缩
  |-- 计算 HMAC
  |-- 对数据与 MAC 加密，并按分组密码规则填充
  `-- 加上 TLS 头部，交给 TCP
```

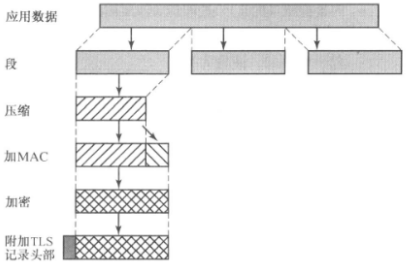


图 17.3　TLS 记录协议的操作过程

图 17.3 TLS 记录协议的操作过程

TLS 使用 HMAC：

$$\mathrm{HMAC}_K() = H\left(K \ \text{opad}\right) \| H\left(K \ \text{ipad}\right) \|$$

对 TLS，MAC 输入为

$$\mathrm{MAC_rite_secret} \| \mathrm{se_nm} \| \mathrm{type} \| \mathrm{version} \| \mathrm{lent} \| \mathrm{rament}$$

其中版本字段也被保护，能发现协议版本被篡改。记录头部包含内容类型、主版本号、从版本号和压缩长度；内容类型可为 `change_cipher_spec`、`alert`、`handshake` 或 `application_data`。

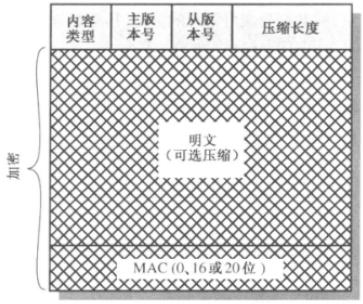


图 17.4　TLS 记录格式

图 17.4 TLS 记录格式

分组密码的填充长度可变并且是分组长度的整数倍，可在一定程度上阻止基于消息长度的分析；流密码则把压缩数据和 MAC 一起加密。TLSv1.2 的教材示例算法包括 AES、3DES、RC4-128，MAC 哈希包括 MD5 或 SHA-1。

2.3 修改密码规范与警报协议

接所用的密码组。

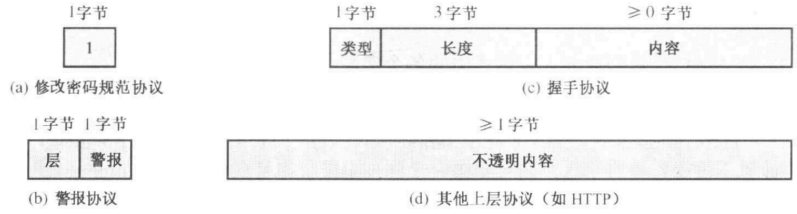


图 17.5　TLS 记录协议净荷

图 17.5 TLS 记录协议净荷

- **修改密码规范协议**：只有一个值为 1 的字节。收到后，挂起的密码状态复制为当前状态，后续记录使用新密码组。
- **警报协议**：每条消息 2 字节。第一个字节为警告（1）或致命（2），第二个字节描述具体错误。致命警报立即终止当前连接，但会话中的其他连接可继续存在。

2.4 握手协议

握手在应用数据传输前执行，用于相互认证、协商版本、密码组、压缩方法、MAC 算法和保护记录的密钥。握手消息由类型（1 字节）、长度（3 字节）和内容组成，常见类型包括 client_hello、server_hello、certificate、server_key_exchange、certificate_request、server_done、client_key_exchange、certificate_verify 和 finished。

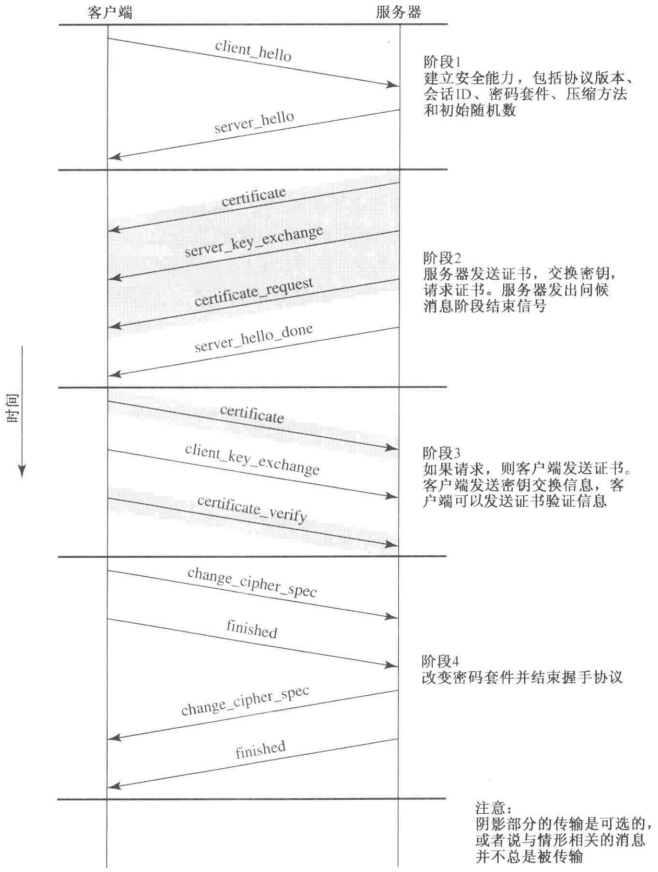


图 17.6　握手协议的处理过程

图 17.6 TLS 握手的四阶段处理过程

四阶段可概括为：

1. **建立安全能力**：客户端发送支持的最高版本、随机数、会话 ID、密码组列表和压缩方法；服务器返回自己的随机数、会话 ID、选定密码组和压缩方法。
2. **服务器认证和密钥交换**：服务器发送 X.509 证书；根据密钥交换方法，可能发送签名的临时 RSA 或 Diffie-Hellman 参数、证书请求和 server_done 。签名哈希包含两个 Hello 随机数和服务器参数，可防重放与伪装。
3. **客户端认证和密钥交换**：客户端验证服务器证书；若被请求则发送自己的证书；发送 client_key_exchange ，并可用 certificate_verify 证明拥有客户端私钥。
4. **完成**：双方依次发送 change_cipher_spec 和加密的 finished ，验证握手和认证过程成功后开始传输应用数据。

支持的密钥交换包括 RSA、固定 Diffie-Hellman、瞬时 Diffie-Hellman 和匿名 Diffie-Hellman。匿名 Diffie-Hellman 不认证参数，容易受到中间人攻击；瞬时 Diffie-Hellman 使用临时参数和签名，在三种 DH 方式中安全性最高。

2.5 TLS 密钥计算与 PRF

TLS 先交换 `pre_master_secret`，再生成共享 48 字节 `master_secret`：

$$\text{master_secret} = \text{PRF}\big(\text{pre_master_secret}, \text{“master secret”}, \text{ClientHello.random} \,\|\, \text{ServerHello.random}\big)$$

随后生成密钥块：

$$\text{key_block} = \text{PRF}\big(\text{master_secret}, \text{“key expansion”}, \\ \text{server_random} \,\|\, \text{client_random}\big)$$

从密钥块按顺序取出客户端/服务器写 MAC 密钥、写加密密钥和 IV。

TLS 的 PRF 由 HMAC 扩展函数 P_hash 构成：

$$P_hash(\text{secret}, \text{seed}) = \text{HMAC}(\text{secret}, A(1) \,\|\, \text{seed}) \\ \|\, \text{HMAC}(\text{secret}, A(2) \,\|\, \text{seed}) \,\|\, \cdots$$

其中 $A(0) = \text{seed}$，$A(i) = \text{HMAC}(\text{secret}, A(i-1))$，并且

$$\text{PRF}(\text{secret}, \text{label}, \text{seed}) = P_hash(\text{secret}, \text{label} \,\|\, \text{seed})$$

PRF 以较短共享密钥扩展出任意长度伪随机数据（为了生成密钥或验证）；标签和随机数把不同用途、不同连接的输出分隔开。TLSv1.2 采用基于 SHA-256 的 PRF，教材同时说明早期方案可组合 MD5 和 SHA-1。

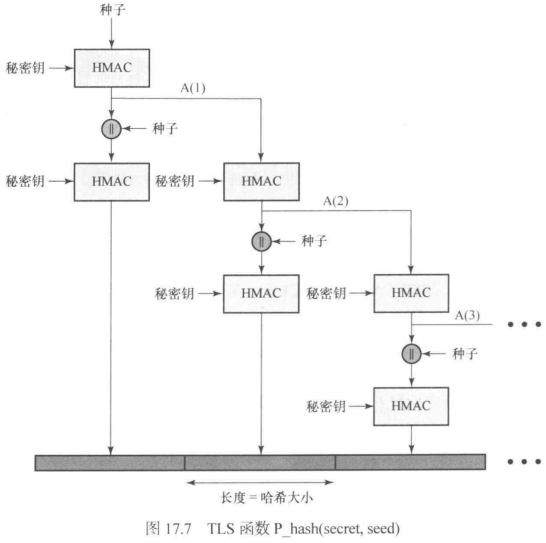


图 17.7　TLS 函数 P_hash(secret, seed)

图 17.7 TLS 函数 P_hash(secret, seed)

2.6 SSL/TLS 攻击与 TLSv1.3 改进

攻击可分为四类：握手协议攻击；记录和应用数据攻击（如 BEAST、CRIME）；PKI/证书验证攻击；以及利用握手和重新协商开销实施的 DoS 攻击。TLS 的发展体现为攻击与补丁持续推动协议修订，正确实现和证书验证同样重要。

教材列出的 TLSv1.3 改进包括：

- 删除压缩、不提供认证加密的密码、静态 RSA/DH、Hello 中 32 位时间戳、重新协商、修改密码规范协议、RC4，以及 MD5/SHA-224 签名；
- 使用 DH 或 ECDH，禁用 RSA 密钥交换，从而提供每次握手新的协商密钥和前向安全；
- 通过调整消息顺序把握手缩短到一次往返，减少延迟和攻击面。

3. 学习目标 3：SSL 与 TLS 的不同

SSL 是 TLS 的前身和基础，TLS 是 IETF 标准化后的继承协议。二者的共同核心是：在 TCP 之上提供记录、握手、密码状态切换和警报机制；差异主要体现为标准版本、密码套件、PRF、可用功能和安全性改进。

比较点	SSL/早期 TLS	TLSv1.3 方向
密钥交换	可用 RSA、固定/瞬时/匿名 DH。	只使用 DH 或 ECDH，移除静态 RSA。
密码功能	允许 RC4、旧式 MAC、压缩和重新协商等历史功能。	删除已弃用功能，优先认证加密。
握手	多阶段、多次往返。	一次往返，尽早建立保护密钥。
安全目标	依赖证书、随机数、MAC 和加密组合。	减少攻击面并强化前向安全。

4. 学习目标 4：TLS PRF 与一般伪随机函数的比较

TLS PRF 与第 8 章中的 PRNG/PRF 一样，都是由秘密种子扩展出不可预测的伪随机位；但 TLS PRF 是带上下文的密钥派生函数：

1. 输入包含 `secret`、用途标签 `label` 和种子 `seed`，而不是只输入一个种子。
2. 通过 HMAC 的 $A(i)$ 链和 P_hash 重复产生任意长度输出。
3. ClientHello/ServerHello 随机数、会话标识和用途标签把不同连接、方向和密钥用途分离。
4. 输出直接用于 master secret、MAC 密钥、加密密钥、IV 和 `finished` 验证值，因此要求密钥不可预测、输出不可区分且上下文不能混用。

5. 学习目标 5：HTTPS

HTTPS 是 HTTP over TLS。它为浏览器与网站之间的通信提供：

- **保密性**：保护 URL、文档内容、表单数据、Cookie 和 HTTP 头部；
- **完整性**：检测传输中的有意或无意修改；
- **认证**：通过网站证书确认浏览器连接到预期网站，降低中间人攻击风险。

连接建立流程为：先建立 TCP 连接，再由 TLS ClientHello 启动握手；TLS 握手成功后，客户端发出 HTTP 请求，所有 HTTP 数据作为 TLS `application_data` 传输。HTTPS 同时涉及 HTTP、TLS 和 TCP 三层连接关系：一个 TLS 会话可支持多个连接。

关闭时，双方应通过 TLS 警报协议发送 `close_notify`，再关闭 TCP。只发送单方关闭警报称为不完全关闭；若 TCP 在没有 `close_notify` 和 `Connection: close` 的情况下断开，客户端应产生安全警告，因为攻击者可能利用未通知的截断。

6. 学习目标 6：SSH

SSH2 是替代 TELNET 的安全远程登录和通用安全通信协议，建立在 TCP 之上，由三个协议层组成：

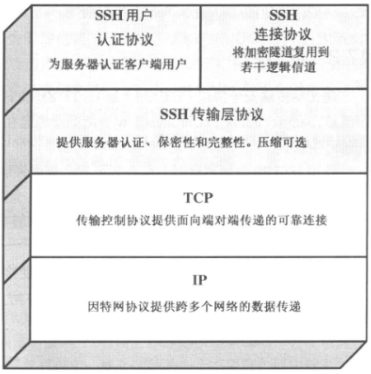


图 17.8 SSH 协议栈

图 17.8 SSH 协议栈

层	作用
SSH 传输层协议	服务器认证、保密性、完整性和可选压缩，并可提供前向安全。
SSH 用户认证协议	让用户向服务器证明身份。
SSH 连接协议	在一条安全低层连接上复用多个逻辑信道。

6.1 SSH 传输层协议

服务器用主机公私钥对认证。客户端可维护“主机名—公钥”本地数据库，也可只信任 CA 根密钥并验证 CA 签发的主机证书；前者无需集中设施但维护开销大，后者减少本地数据库但要求主机密钥均由 CA 认证。

传输层数据包包含数据包长度、填充长度、净荷、随机填充和 MAC。协商加密后，除 MAC 外的整个数据包加密；MAC 覆盖数据包和隐式 32 位序号。协议依次进行标识串交换、 SSH_MSG_KEXINIT 算法协商、Diffie-Hellman 密钥交换、 SSH_MSG_NEWKEYS 切换新密钥和 SSH_MSG_SERVICE_REQUEST 服务请求。



图 17.9 SSH 传输层数据包交换

图 17.9 SSH 传输层数据包交换

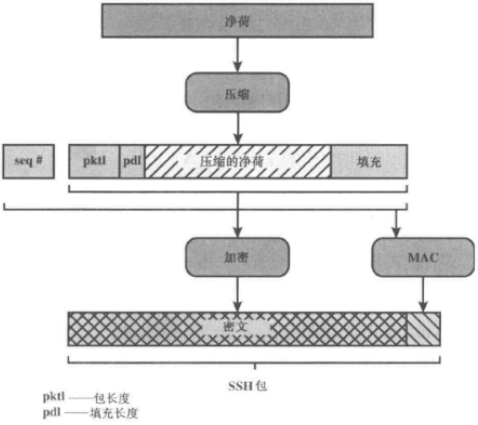


图 17.10 SSH 传输层数据包格式

图 17.10 SSH 传输层数据包格式

Diffie-Hellman 过程中，客户端计算 $e = g^x \bmod p$，服务器计算 $f = g^y \bmod p$，双方得到共享主密钥

$$K = e^y \bmod p = f^x \bmod p$$

服务器用主机私钥签名包含双方标识串、KEXINIT 消息、主机公钥、e、f 和 K 的哈希 H；客户端验证主机公钥和签名。由 K、H 和会话标识派生双向 IV、加密密钥和完整性密钥。

6.2 用户认证与连接协议

用户认证请求包含用户名、服务名和认证方法。服务器可返回失败消息、可继续的方法列表以及“部分成功”标志；所需方法全部成功后返回 `SSH_MSG_USERAUTH_SUCCESS`。

常见认证方法有：客户端公钥签名、在传输层加密保护下发送口令、以及由客户端主机私钥证明主机身份的基于主机认证。基于主机认证并不直接证明用户本人身份，而是信任已认证主机对用户的声明。

连接协议将一条安全认证连接复用为多条逻辑信道。每条信道有本地信道号、窗口大小和最大包长，生命期为开启、数据传输和关闭。信道类型包括会话、X11、前向 TCP/IP 和直接 TCP/IP。

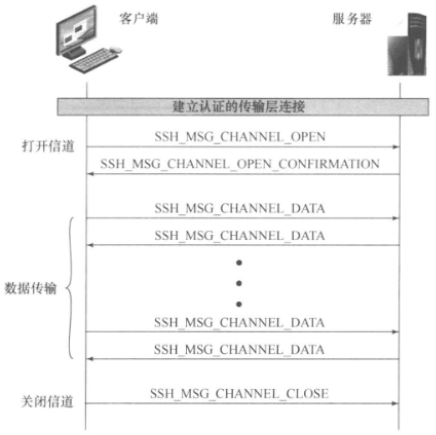


图 17.11　SSH 连接协议消息交换的例子

图 17.11 SSH 连接协议消息交换

6.3 SSH 端口转发

端口是 TCP 用户的标识。SSH 可把不安全 TCP 连接转换为安全隧道：客户端应用端口 x 的流量被重定向到本地 SSH，再经过客户端—服务器之间的 SSH 传输层隧道，由远程 SSH 转发到服务器应用端口 y 。

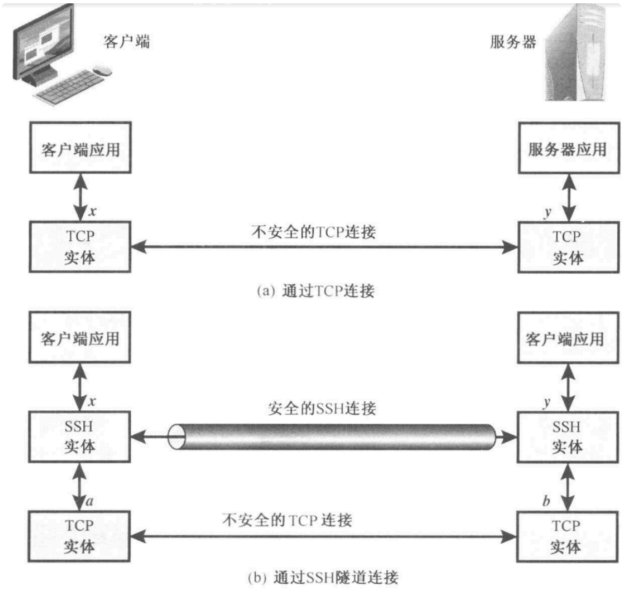


图 17.12　SSH 传输层数据包交换

图 17.12 SSH 端口转发

本地转发由客户端建立监听端口，截获应用流量并送入 SSH 隧道；远程转发则由服务器侧监听并把流量经隧道送回客户端网络。两者共同作用是让原本没有密码保护的 TCP 应用借助 SSH 获得保密性、完整性和服务器认证。

> **关于 SSH 与 TCP 之间的关系的理解**
>
> SSH 协议建立在 TCP 协议之上，客户端与服务器之间的所有原始数据均通过同一条 TCP 连接进行可靠传输。然而，单条 TCP 连接本质上是一个有序的字节流，无法区分同时进行的多个独立业务（如交互式 Shell、文件传输、端口转发）。为解决这一问题，SSH 协议栈在传输层之上实现了**信道（Channel）机制**。
> 在通信过程中，每个信道被分配独立的信道号，发送端将所有业务数据封装为携带信道标识的数据包，通过同一条 TCP 连接复用传输；接收端根据数据包中的信道号将其拆分并分发给对应的处理进程。这一机制实现了三大核心功能：
>
> 1. **多路复用**：单条 TCP 连接可同时承载多个逻辑信道，避免了为每个业务重复建立 TCP 连接的开销。
> 2. **精细流控制**：每个信道拥有独立的窗口，可分别调节发送速率，避免某一大流量业务阻塞其他交互式业务。
> 3. **资源隔离**：不同信道的流量相互独立，某个信道的拥塞或异常不会影响其他信道的正常通信。
>
> 总而言之，TCP 提供了传输管道，SSH 信道则提供了管道内的逻辑隔离与业务复用能力。

本章关键术语整理

术语	核心含义
TLS	在 TCP 之上提供认证、保密性和完整性的通用安全协议组。
SSL	TLS 的前身，已被新 TLS 标准取代。
TLS 记录协议	对上层数据执行分段、压缩、MAC、加密和封装的协议。
TLS 握手协议	协商版本、密码组、认证方式和密钥的协议。
TLS 会话	可由多个 TLS 连接共享密码参数的关联。
TLS 连接	TLS 对等实体之间短暂的安全传输关系。
密码规范	定义加密算法、MAC、密钥材料、IV 等参数的状态。
PRF	由秘密、标签和种子扩展出任意长度伪随机输出的函数。
HTTPS	运行在 TLS 上的 HTTP，提供 Web 通信的保密性、完整性和认证。
SSH	提供安全远程登录、数据传输、认证和端口转发的协议组。
SSH 主机密钥	服务器用于向客户端证明主机身份的公私钥对。
SSH 传输层协议	提供服务器认证、加密、完整性和可选压缩的 SSH 子协议。
SSH 用户认证协议	使用户向服务器证明身份的 SSH 子协议。
SSH 连接协议	在安全低层连接上复用多个逻辑信道的 SSH 子协议。
SSH 隧道	通过 SSH 安全连接转发原本不安全 TCP 流量的机制。

学习目标自检

1. 能否按完整性、保密性、可用性和认证说明 Web 威胁、后果与对策？
2. 能否比较 IPsec、TLS 和应用层安全在 TCP/IP 协议栈中的位置及取舍？
3. 能否区分 TLS 会话、TLS 连接、当前状态和挂起状态？

4. 能否写出 TLS 记录协议的处理顺序，并说明 MAC 覆盖哪些字段？
5. 能否说明修改密码规范协议、警报协议和握手协议分别承担什么职责？
6. 能否按四阶段描述 TLS 握手，并说明 RSA、固定 DH、瞬时 DH 和匿名 DH 的认证差异？
7. 能否写出 master_secret、key_block 和 PRF/P_hash 的核心公式？
8. 能否比较 SSL、早期 TLS 与 TLSv1.3 在密钥交换、密码功能和握手往返次数上的差异？
9. 能否说明 HTTPS 如何初始化和关闭 TLS 连接，以及为何需要 close_notify？
10. 能否列出 SSH 三个协议层的职责，并说明主机密钥如何认证服务器？
11. 能否描述 SSH 传输层的标识交换、算法协商、DH 密钥交换、NEWKEYS 和服务请求？
12. 能否区分 SSH 用户认证方法、连接信道和本地/远程端口转发？